

Asignatura: Fundamentos de algoritmia

Evaluación continua. Cuestionario.

Grupo A (azul).
Profesor: Isabel Pita.

Nombre del alumno:

1. Dadas las siguientes funciones de coste, indica su orden de complejidad (representado por la función más sencilla) y ordena los órdenes de complejidad obtenidos utilizando $\subset$.

- a) $f_1(n) = 6$
- b) $f_2(n) = 4n + \log n^2$
- c) $f_3(n) = n^5 + 3n^3 + 2$
- d) $f_4(n) = 2^n + 3^n$
- e) $f_7(n) = 3n \log 5n + \log n$

Respuesta:

- a) $f_1(n) = 6 \in \mathcal{O}(1)$
- b) $f_2(n) = 4n + \log n^2 \in \mathcal{O}(n)$
- c) $f_3(n) = n^5 + 3n^3 + 2 \in \mathcal{O}(n^5)$
- d) $f_4(n) = 2^n + 3^n \in \mathcal{O}(3^n)$
- e) $f_7(n) = 3n \log 5n + \log n \in \mathcal{O}(n \log n)$

$$\mathcal{O}(1) \subset \mathcal{O}(n) \subset \mathcal{O}(n \log n) \subset \mathcal{O}(n^5) \subset \mathcal{O}(3^n)$$

2. Dadas dos matrices m_1 y m_2 de dimensiones $m_1 \times m_2$ y $m_2 \times m_3$, indica la complejidad en tiempo más ajustada del siguiente algoritmo, y justifícala indicando el número de vueltas que da cada bucle y el coste de cada vuelta.

```
for (int i = 0; i < m1; ++i) {  
    for (int j = 0; j < m3; ++j) {  
        int suma = 0;  
        for (int k = 0; k < m2; ++k)  
            suma += m1[i][k] * m2[k][j]  
        p[i][j] = suma;  
    }  
}
```

Respuesta:

El bucle interno (variable de control **k**) da **m2** vueltas y cada vuelta tiene coste constante porque consiste en dos operaciones aritméticas y dos accesos a la matriz, todas ellas operaciones de coste constante. Por lo tanto el coste del bucle interno es del orden de **m2**.

El bucle interno con variable de control **j** da **m3** vueltas. Cada vuelta tiene coste $\max(c_0, m_2, c_1)$, porque se trata de tres instrucciones en secuencia, la primera tiene coste constante por ser una asignación, la segunda es un bucle de coste **m2**, y la tercera es de coste constante por ser una asignación. El coste de cada vuelta es por lo tanto **m2**. Como el bucle da **m3** vueltas, el coste de este bucle es **m2 * m3**.

El bucle externo da **m1** vueltas. Cada vuelta tiene coste **m2 * m3**. Por lo tanto el coste completo del algoritmo es **m1 * m2 * m3**.

3. Escribe un predicado para expresar que todos los valores de un vector v son diferentes.

Respuesta:

$$diferentes(v) \equiv \forall x, y : 0 \leq x < y < v.size() : v[x] \neq v[y].$$

4. Escribe un predicado que dado un vector v y dos índices del vector i, j , indique si entre los valores correspondientes a los índices del vector existe una inversión. Entre dos valores existe una inversión si el primero es mayor que el segundo.

Respuesta:

$$Inversion(v, i, j) \equiv v[i] > v[j]$$

5. Especifica un algoritmo que dado un vector de números enteros, todos ellos diferentes, cuya longitud es mayor que cero calcule el número de inversiones del vector.

Respuesta:

En este caso nos piden la especificación completa de un algoritmo. Debemos dar la precondition y la postcondición.

$$P : \{v.size() > 0 \wedge diferentes(v)\}$$

`inversiones(vector<int> v< > dev int cont`

$$Q : \{cont == \#p, q : 0 \leq p < q < v.size() : inversion(v, p, q)\}$$

6. Dada la especificación:

$$P: \{v.size() \geq 0\}$$

`resolver(vector<int> v) dev int np`

$$Q: \{np == \max p, q : 0 \leq p \leq q \leq v.size() \wedge secPositiva(v, p, q) : q - p\}$$

Y dada la siguiente implementación:

```
int resolver(std::vector<int> const& v) {
    int np = 0; int longAct = 0;
    for (int i = 0; i < v.size(); ++i) {
        if (v[i] > 0) { // El elemento continua la racha
            ++longAct;
            if (np < longAct) np = longAct;
        }
        else longAct = 0; // Se rompe la racha
    }
    return np;
}
```

Indica un invariante del bucle que nos permita probar la corrección del mismo. Recuerda que en el invariante se debe expresar lo que se calcula en todas las variables que intervienen en el bucle, no solo en la variable que se devuelve como resultado de la función.

Respuesta:

La primera parte del invariante indica que `np` guarda la longitud del máximo segmento de valores positivos encontrado hasta la posición `i`.

La segunda parte del invariante indica que `longAct` guarda la longitud del segmento máximo de números positivos que termina en la posición `i`.

$$I : \{np == \max p, q : 0 \leq p \leq q \leq i \wedge secPositiva(v, p, q) : q - p\} \wedge \\ longAct = \min p : 0 \leq p \leq i \wedge secPositiva(v, p, i) : i - p\}$$

7. Dado un vector de números enteros, cuyos elementos son todos diferentes, y no están ordenados, indica un algoritmo que permita obtener un valor del vector que no sea ni el máximo ni el mínimo en tiempo constante.

Respuesta:

Si el vector tiene menos de tres elementos el valor no existe.

Si el vector tiene tres o mas elementos, se cogerán los tres primeros, correspondientes a las posiciones `v[0]`, `v[1]` y `v[2]`. De estos tres valores se rechazan el máximo y el mínimo (que tienen posibilidad de ser el máximo o minimo del vector) y se devuelve el que queda.

8. Indica cuál es la recurrencia que permite calcular el coste en el caso peor del siguiente algoritmo, haz el desplegado e indica el coste final del algoritmo.

```

lli f(std::vector<int> & v, int ini, int fin) {
    if (fin <= ini) return 0;
    else {
        int m = (ini + fin) / 2;
        lli iz = f(v, ini, m);
        lli dr = f(v, m+1, fin);
        lli aux = 0;
        int i = ini, j = m+1;
        while (i <= m && j <= fin) {
            if (v[i] > v[j]) {
                aux += (m-i+1);
                ++j;
            }
            else ++i;
        }
        std::inplace_merge(v.begin(), v.begin()+m, v.begin()+fin);
        return iz + dr + aux;
    }
}

```

Respuesta:

Se realizan dos llamadas recursivas cada una con la mitad de los datos del vector. La parte recursiva del algoritmo excluyendo las llamadas recursivas tiene coste n siendo n el número de elementos del vector, debido a que se recorre el vector en el bucle while realizando una operación de coste constante para cada elemento que se trata del vector y la operación `inplace_merge` tiene también coste lineal en el número de elementos del vector.

$$T(n) = \begin{cases} c_0 & \text{if } ini == fin \\ 2T(n/2) + n & \text{if } ini < fin \end{cases}$$

El despliegado y el coste final se encuentran en el cuaderno de problemas de complejidad de algoritmos recursivos del campus.

9. Escribe una función recursiva que permita resolver el problema de comprobar si en un vector algún valor coincide con su posición en el vector.

$$coincide(v, ini, fin) = \begin{cases} false & \text{if } ini == fin \text{ (vector vacío)} \\ v[m] == m & \text{if } m = (ini + fin)/2 \\ coincide(v, ini, m) || coincide(v, m + 1, fin) & \text{if } ini + 1 < fin \wedge m = (ini + fin)/2 \end{cases}$$

La llamada inicial a la función es `suma(v, 0, v.size())`.

10. Indica cuál es la mejor complejidad que puede tener un algoritmo de ordenación basado en comparaciones, si los valores del vector son números enteros cualesquiera. Indica si conoces algún algoritmo con esta complejidad y en caso afirmativo di cuál o cuáles son.

Respuesta:

La mejor complejidad que se puede obtener con un algoritmo de ordenación basado en comparaciones es $\mathcal{O}(n \log n)$.

Se han estudiado los algoritmos de quicksort y mergesort con esta complejidad. Quicksort tiene complejidad $\mathcal{O}(n \log n)$ en el caso medio y complejidad $\mathcal{O}(n^2)$ en el caso peor.

Mergesort tiene complejidad $\mathcal{O}(n \log n)$ en el caso peor y en el caso medio.